

NSL Cross-Paper Connections and Refinements: Compositional Wins and Individual Paper Adjustments

Status: Research paper (cross-paper analysis; composition optimizations and paper-level refinements) **Date:** 2026-04-16 **Audience:** Implementers considering which cross-paper composition to pick up next; reviewers assessing whether shipped papers need revision; contributors looking for high-impact low-cost work

Abstract

The April 2026 cross-paper audit of NSL's fourteen shipped technique papers identified seven unexploited compositions — optimizations enabled by pairing two or more existing techniques that no shipped implementation takes advantage of — and five individual-paper refinements that would improve the shipped papers themselves. This paper documents both sets.

The seven compositions are: **C-01** FASE freed-gradient memory feeding CFIP's tile-size budget; **C-02** CSHA Tier C save-activations stored in a CFIP-in-HBM format; **C-03** CPDT all-reduce of already-CFIP-quantized gradients; **C-04** WRGA dead-head gradients using reduced CFIP precision; **C-05** PCA segment boundaries informing CSHA tile scheduling; **C-06** WGGO per-layer sensitivity feeding CFIP format selection; **C-07** BYOD datatype mechanism unifying with CFIP's format declarations.

The five refinements are: **R-01** CFIP's type-system extension reformulated as an effect rather than an AST-level residency enum; **R-02** grad-context bridge behavior specified for MoE routing and speculative decoding; **R-03** @export's interaction with interior quantization formats specified; **R-04** FASE's AdamW bias correction fix elevated from implementation discovery to a numbered paper appendix; **R-05** WRGA's B.3 scaffolding retrospective integrated into the WRGA paper itself.

Each composition is estimated at 1–3 implementation sessions; each refinement is primarily a documentation update but some have minor code changes. The paper concludes with an implementation priority matrix and a testing strategy that catches the cross-technique regressions these compositions could introduce.

§1 Introduction

§1.1 Context

The fourteen NSL technique papers were each completed in isolation. Each paper was reviewed against its own research question and against the NSL codebase; cross-paper interactions were considered in passing during review but not systematically audited. The composition paper (2026-04-16) introduced the discipline of mapping every technique onto pipeline stages and listing cross-technique invariants. That exercise surfaced a set of compositions the shipped implementations do not yet exploit — cases where two techniques' effects compound but no code path currently bridges them.

Separately, the audit identified five individual-paper issues: design choices that were reasonable when the paper was written but which the composition view suggests should be revised. These range from a technical redesign (R-01) to documentation of previously-implicit behavior (R-02, R-03) to elevating implementation-time discoveries into paper-level institutional memory (R-04, R-05).

§1.2 Scope

The compositions described here are *extensions* to existing techniques — each is implementable without changing any technique's external contract. A consumer of CFIP does not need to know that its input was produced by a FASE-freed memory buffer; a consumer of a CSHA-saved activation does not need to know it is in a CFIP-in-HBM format because the dequant-on-read makes it transparent. This is the property that makes the compositions low-risk.

The refinements are revisions to shipped papers. R-01 involves code changes; R-02, R-03 are specification additions with small code impact; R-04, R-05 are documentation-only updates to papers' institutional-memory appendices.

§1.3 Organization

§2 through §8 cover the seven compositions, one per section. §9 through §13 cover the five refinements, one per section. §14 is the priority matrix and recommended sequencing. §15 is the testing strategy for cross-technique regressions.

§2 Composition C-01 — FASE freed-gradient memory feeding CFIP's tile-size budget

§2.1 Observation

FASE Deferred mode eliminates the per-parameter gradient buffer. For a 150M-parameter model in FP16 gradients, this is approximately 300MB of memory freed. For a 7B-parameter model, 14GB. The memory is freed at runtime when FASE's Deferred path is taken: the buffer allocation simply does not occur.

CFIP's tile-size selection operates against a fixed SMEM budget (the SM's 228KB H100, 256KB Blackwell) and a per-kernel register budget. Interior-format assignments are chosen to maximize the effective tile size. Tile size affects attention kernel throughput substantially (larger tiles → fewer kernel launches → better overlap and better arithmetic intensity).

The freed gradient-buffer memory is HBM memory, not SMEM, so it does not directly feed tile-size selection. But it does affect which *larger-staging* designs become feasible. CSHA Tier A, for instance, requires holding the entire attention layer's activations in HBM between forward and backward — if HBM is tight, Tier A is ruled out even when it would be fastest. FASE's Deferred-mode savings free HBM that could unlock Tier A selections where previously Tier B was forced.

More subtly: CSHA's save-activations can be written to freed gradient-buffer pages if CFIP and M36 cooperate. The page allocator currently treats gradient-buffer HBM and activation HBM as separate pools. With a small change to M36 — a mode where the gradient-buffer pages are reassigned to the activation pool when FASE is Deferred — the effective activation budget grows.

§2.2 Optimization

M36's memory planner gains a `free_pool_source` field that records which technique's elimination freed which memory region. FASE Deferred publishes "gradient buffer pages: N bytes, available from step T." CFIP's tile selector and CSHA's tier selector both read the free-pool state.

WGGO's decision table gains a "post-FASE memory budget" entry that downstream decisions consult. CSHA's tier-selection heuristic, currently based on attention shape alone, now reads the post-FASE memory budget. CFIP's per-kernel tile-size target is unchanged (SMEM is the constraint) but CFIP-in-HBM save-activations (see §3) gain the freed memory as additional working room.

§2.3 Estimated impact

For models at the margin where FASE freed memory crosses the Tier A / Tier B threshold, the impact is large (~10-20% training throughput improvement because Tier A's recompute-free backward is measurably faster than Tier B's explicit-backward). For models far from the margin, the impact is zero. In typical 7B-parameter training at 1024-token sequences on an H100, early analysis suggests 30-40% of layers would benefit from Tier A when FASE Deferred is active and would otherwise take Tier B.

§2.4 Implementation scope

- M36: add `free_pool_source` field, reporting from FASE Deferred. ~150 LOC.
- WGGO: extend the decision table with the post-FASE memory budget column; modify CSHA tier selection to read it. ~200 LOC.
- Testing: canonical models at memory thresholds; verify tier transitions track FASE mode. ~300 LOC.

Total: ~650 LOC. Estimated 1-2 implementation sessions.

§3 Composition C-02 — CSHA Tier C save-activations in CFIP-in-HBM format

§3.1 Observation

CSHA Tier C writes Q_{proj} , K_{proj} , V_{proj} , row_max , and row_sum to HBM during forward. These are read-once (during backward) by a separately-generated kernel. The HBM write cost is a non-trivial share of attention-layer HBM traffic: on a 1024-token sequence with $d_{\text{model}}=1024$ and 8 heads, the save-activation write is approximately 40MB per layer. Over 6 layers this is 240MB per forward pass.

The current format is FP16. Writing FP16 is twice the bandwidth of FP8; writing a 4-bit interior-format representation is four times less than FP16.

CFIP's IF4, IL2, and IM3 formats are designed as kernel-interior formats — they live in SMEM and registers, not HBM. The reason is generality: any tensor crossing a kernel boundary could be consumed by a kernel with different precision assumptions, so boundary tensors must be in a well-known external format. But CSHA save-activations cross a specific, well-known boundary: from a specific forward kernel to a specific backward kernel, with no intermediate consumer. The locality is much stronger than general HBM traffic.

A restricted CFIP format — **CFIP-in-HBM** — would be legal for this specific use case: a named pair (forward kernel, backward kernel) agrees at compile time to use a CFIP format for the save-activation buffer. The format is stored as metadata on the buffer; the backward kernel's dequant-on-read matches the forward's format-on-write. No other code path can read these buffers.

§3.2 Optimization

CFIP gains a "semi-interior" format category: a CFIP format tagged as legal in HBM when the tensor's producer and consumer are both named and compile-time-known. CSHA Tier C's save-activation buffer qualifies. IF4 is used for Q_{proj} , K_{proj} , V_{proj} (dense linear projections with roughly Gaussian statistics). IM3 is used as a fallback if the symbolic analysis shows IF4 is outside its envelope. row_max and row_sum remain in f32 (they are reduction results with accumulated-error characteristics that reject lower precision).

The effective HBM write reduction for save-activations:

- $Q_{\text{proj}}/K_{\text{proj}}/V_{\text{proj}}$ at IF4: 4× reduction from FP16.
- row_max , row_sum at f32: unchanged.
- Net weighted reduction: approximately 3× for the save-activation buffer (the projections dominate the bytes).

Over the entire forward pass, save-activation HBM traffic is ~30% of total attention HBM traffic. A 3× reduction on that share is approximately a 20% reduction in total attention HBM traffic, which translates to 5-10% training throughput improvement (attention is typically 30-40% of training compute at typical sizes).

§3.3 Estimated impact

5-10% training throughput improvement on the canonical 7B-parameter Llama-style architecture. Larger impact for smaller models (attention is a bigger share). Smaller impact for MoE models (where expert compute dominates).

§3.4 Implementation scope

- CFIP: add semi-interior format category, per-tensor HBM-eligibility flag. ~200 LOC.
- CSHA Tier C: save-activation buffer gains a format field; forward kernel emits in the chosen format; backward kernel dequants on read. ~400 LOC across forward and backward emitters.
- M36 memory planner: allocate save-activation buffers sized for the chosen format. ~100 LOC.
- Testing: numerical equivalence between CFIP-in-HBM Tier C and FP16 Tier C across a matrix of attention shapes. ~500 LOC.

Total: ~1200 LOC. Estimated 3-4 implementation sessions. Higher LOC reflects the numerical-testing burden.

§4 Composition C-03 — CPDT all-reduce of already-CFIP-quantized gradients

§4.1 Observation

Gradients produced by backward kernels are emitted in CFIP interior formats when the backward kernel uses CFIP (typically IL2 or IM3 for accumulators). Currently, on the way out of the backward kernel, they are dequantized to f32 before being written to HBM. At HBM, they are in f32. CPDT's all-reduce then reads them in f32, sums across ranks, and writes the sum back in f32.

If the all-reduce happened directly in the CFIP format, network bandwidth would drop by the format's compression ratio: 4× for IF4, 5.3× for IL2 (2 bits effective), 10.7× for IM3 (3 bits effective). Since inter-node bandwidth is the binding constraint for multi-node training at scale, this has direct training-throughput impact.

The technical challenge is associativity. All-reduce requires the aggregation operation to be associative so that a ring-all-reduce is mathematically equivalent to a naive all-sum. For floating-point formats, sum is approximately associative; for fixed-point formats, sum is exactly associative until overflow. CFIP formats are fixed-point-like (integer mantissa + block scale). Summing 8 ranks' per-block contributions is well-defined until the block's accumulated value exceeds its representable range.

The solution is block-scale renormalization: after summing, if any block's values exceed the representable range, the block's scale is adjusted. This happens lazily — most blocks stay within range. For the rare block that overflows, the renormalization step is a per-block scalar multiply.

§4.2 Optimization

CPDT gains a "compressed all-reduce" path. The path is selected per-layer when:

- The backward kernel produces gradients in a CFIP format.
- The CPDT plan specifies that layer is in a data-parallel shard (not just tensor/pipeline parallel).
- The network-bandwidth estimate suggests the reduction is bandwidth-limited (not compute-limited).

The path reads gradients in the CFIP format from the backward's output SMEM/registers, writes them to HBM in the compressed format, invokes a CFIP-aware all-reduce primitive (typically NCCL with a custom reduction op), and leaves the result in the compressed format for the optimizer step. The optimizer step dequants on read.

This composes with FASE Deferred: when FASE is Deferred, the reduce target is MHat/VHat 's gradient contribution, which stays compressed. When FASE is Immediate, the reduce target is the gradient buffer, which is compressed instead of f32.

§4.3 Estimated impact

On multi-node training with 10Gbps inter-node bandwidth (typical commodity), current NCCL all-reduce consumes 30–50% of training step time. A $4\times$ bandwidth reduction (using IF4 for most layers) brings this to 7–12%, improving total training throughput by 20–40%.

At faster inter-node bandwidth (200Gbps NVLink-accelerated InfiniBand), the benefit shrinks to 5–15% because communication is less of a share.

§4.4 Implementation scope

- CPDT: compressed all-reduce path with format-aware reduction. ~600 LOC.
- NCCL integration: custom reduction op registered with NCCL. ~300 LOC.
- Block-scale renormalization logic. ~200 LOC.
- FASE integration (compressed reduction into MHat/VHat). ~150 LOC.
- Testing: multi-rank training with and without compression; verify convergence parity. ~600 LOC.

Total: ~1850 LOC. Estimated 6-8 implementation sessions. Requires multi-node test environment.

Note: This is closely related to gap 2 (CGAC, gradient compression for all-reduce). The difference is that CGAC is a separate technique that chooses a compression algorithm (PowerSGD, QSGD, etc.); C-03 is opportunistic use of existing CFIP formats. Both could coexist: CGAC for gradients not in CFIP format, C-03 for gradients already in CFIP format.

§5 Composition C-04 — WRGA dead-head gradients with reduced CFIP precision

§5.1 Observation

WRGA's Tier C dead-head analysis (deferred from B.3 scaffolding; listed in the WRGA project file) identifies attention heads whose gradients are approaching zero and scheduled for pruning. Before full pruning, a head's gradients shrink toward zero but retain small non-zero contributions. Computing those gradients at full CFIP precision wastes computation on values that are about to be zeroed.

CFIP's bit allocation is currently static per-layer. A dynamic per-head allocation based on the head's "death probability" would give near-zero-gradient heads the lowest precision (IF4 floor of 2 bits) and preserve higher precision (IL2, IM3) for active heads.

The death probability is a scalar per head that updates each training step based on the head's gradient-norm trajectory. When death probability exceeds a threshold (e.g., 0.9), the head's CFIP interior format shifts to the lower-precision path. When it drops below, the format returns to normal.

§5.2 Optimization

WRGA emits a per-head death probability into a runtime-visible buffer. CFIP's format selector, currently static at compile time, gains a dynamic re-selection path gated on per-head metadata. The selector reads the death probability per kernel invocation; if it exceeds the threshold, the kernel uses a lower-precision format for that head's accumulator.

This is a small but unusual composition: CFIP's format selection is normally purely compile-time; this adds a small runtime component. The compile-time machinery commits to "IL2 or IF4 per head; decision at runtime." The runtime machinery decides which per kernel invocation. The bit-level format table is the same for both paths.

§5.3 Estimated impact

Small in absolute terms (~1-2% training throughput improvement) but smooth degradation with no loss of accuracy — the precision reduction happens only where the head is going to be zeroed anyway. The benefit compounds with aggressive pruning schedules: a schedule that prunes 40% of heads over training sees roughly 40% of attention backward computation at reduced precision.

§5.4 Implementation scope

- WRGA: per-head death probability buffer; update logic each backward. ~200 LOC.
- CFIP: runtime format-selection path gated on per-head metadata. ~300 LOC.
- Testing: equivalent accuracy vs fully-precision WRGA; benchmark throughput improvement. ~400 LOC.

Total: ~900 LOC. Estimated 2-3 implementation sessions. Low priority because the gain is small; higher priority once aggressive head pruning becomes the standard WRGA config.

§6 Composition C-05 — PCA segment boundaries informing CSHA tile scheduling

§6.1 Observation

PCA (segment-masked attention) knows segment boundaries at compile time if `segment_ids` is constant-foldable, or at per-batch runtime otherwise. CSHA tiles attention into blocks of (`tile_M`, `tile_N`) Q-rows $\times$ K-columns. A tile that spans a segment boundary must apply the segment mask; a tile that is entirely within one segment needs no masking.

Currently CSHA's tile scheduler does not read segment boundaries. All tiles are emitted with the same mask-checking prologue. For tiles that are entirely within one segment, this prologue is wasted overhead (~2-5% of per-tile runtime on masked-attention kernels).

A segment-aware tile scheduler would emit two variants of the attention kernel: one with masking (for tiles crossing boundaries), one without (for tiles within segments). At tile-launch time, the scheduler picks the appropriate variant based on the tile's (`q_start`, `q_end`, `k_start`, `k_end`) range and the segment boundaries.

§6.2 Optimization

CSHA tile scheduler reads a compile-time-emitted segment-boundary table (from PCA). The table maps (`q_start`, `q_end`) $\rightarrow$ list of segments covered. For tiles covering one segment, the no-mask variant is used; for tiles covering multiple, the masked variant.

Tile-variant emission is modest: two kernels are emitted (masked, unmasked) per attention layer. The scheduler picks. The memory cost is a small constant (two kernel entries in the binary); the runtime cost is a per-tile table lookup.

§6.3 Estimated impact

2-5% reduction in per-tile attention runtime on segment-masked layers. Overall training throughput improvement is small (1-2%) because only segment-masked layers benefit, and even within those, only single-segment tiles benefit. But the optimization is essentially free — the scheduler decision is microseconds per tile.

§6.4 Implementation scope

- PCA: emit segment-boundary table as a compile-time artifact. ~200 LOC.
- CSHA: emit masked and unmasked kernel variants; scheduler picks. ~400 LOC.
- Testing: equivalence between masked-everywhere and scheduler-driven paths. ~300 LOC.

Total: ~900 LOC. Estimated 2-3 implementation sessions.

§7 Composition C-06 — WGGO per-layer sensitivity feeding CFIP format selection

§7.1 Observation

WGGO makes per-layer decisions: prune-or-not, CSHA tier, adapter rank, CFTP-or-not. Some of these decisions imply a per-layer sensitivity ranking: a layer WGGO marks for aggressive pruning is low-sensitivity; a layer marked "keep dense" is high-sensitivity. Currently this sensitivity is implicit — it drives WGGO's own decisions but is not exposed to downstream techniques.

CFIP's format assignment operates purely on tensor role and symbolic GEBA bound. Two layers with the same role and same GEBA bound get the same CFIP format, regardless of sensitivity. This is a missed signal.

Exposing WGGO's per-layer sensitivity as a first-class field would let CFIP bias its format selection: for high-sensitivity layers, choose less-aggressive formats (IM3 over IL2, IL2 over IF4) even when GEBA would permit the more aggressive option. For low-sensitivity layers, the GEBA-permitted choice stands.

§7.2 Optimization

WGGO's decision table gains a per-layer `sensitivity` field with values `low`, `medium`, `high`. The field is derived from WGGO's other decisions (e.g., marked-for-pruning → low sensitivity; adapter-rank ≥ 32 → high sensitivity; etc.). CFIP's format selector reads the sensitivity and applies a format-bias adjustment: high → one step more conservative than GEBA-recommended; low → one step more aggressive.

This is a simple machine-learning-adjacent technique: the sensitivity field is an effective covariate for the format selector. The GEBA bound remains the hard constraint; the sensitivity is a preference within the feasible region.

§7.3 Estimated impact

Improves the precision-vs-accuracy tradeoff at the layer level. For models with high heterogeneity across layers (e.g., mixed architectures, layers with different roles), the impact can be 10-30% reduction in overall precision cost at equivalent accuracy. For homogeneous transformer stacks, the impact is smaller (~5%).

§7.4 Implementation scope

- WGGO: derive per-layer sensitivity from existing decisions. ~200 LOC.
- CFIP: format-selector bias logic. ~150 LOC.
- Testing: accuracy parity with explicit per-layer tuning. ~300 LOC.

Total: ~650 LOC. Estimated 1-2 implementation sessions.

§8 Composition C-07 — BYOD datatype mechanism unifying with CFIP formats

§8.1 Observation

NSL's BYOD (Bring Your Own Datatype) system supports user-declared types like `(datatype Ternary: bits: 2, layout: ...)`. The BYOD system is a declarative way to introduce new precision formats. CFIP's IF4, IL2, and IM3 are currently compiler intrinsics — they are hardcoded in the CFIP pass and not expressible in user code.

There are two parallel systems for precision formats. This is a design duplication. A unified system where CFIP formats are expressed as BYOD declarations would:

- Eliminate the duplication.
- Make CFIP extensible without compiler changes (a user could declare their own interior format for a novel use case).
- Simplify the compiler by reducing the number of format-handling code paths.

§8.2 Optimization

CFIP's IF4, IL2, IM3 are reimplemented as BYOD datatype declarations living in a standard library module (e.g., `(nsl.std.cfip_formats)`). The CFIP analysis pass reads format properties (bit width, scale type, representable range) from the BYOD declaration rather than from hardcoded tables. The PTX emitter for CFIP formats reads the same declarations.

User-declared interior formats become possible: `(datatype MyInteriorFormat: bits: 5, scale: f16, layout: [packed, tile=(16,16)])`. The CFIP analysis pass treats them identically to the stdlib formats.

The refactor is invisible to existing users; it's a backend consolidation.

§8.3 Estimated impact

No direct performance impact. The benefit is compiler maintainability and extensibility. This is tier-2 infrastructure work; valuable for long-term but not urgent.

§8.4 Implementation scope

- BYOD type-system extensions: representable-range inference, scale-type field, layout hints. ~400 LOC.
- CFIP pass: read formats from BYOD declarations. ~300 LOC.
- stdlib: declare IF4, IL2, IM3 as BYOD types. ~150 LOC.
- PTX emitter: format-specialization driven by BYOD metadata. ~300 LOC.
- Testing: numerical parity between pre- and post-refactor implementations. ~500 LOC.

Total: ~1650 LOC. Estimated 4-6 implementation sessions. Classic refactor — mechanical, validated by tests.

§9 Refinement R-01 — CFIP type-system extension via effects, not residency enum

§9.1 Issue

The CFIP paper (§3.4) adds a `Residency` enum to `TypeExprKind::Tensor`:

```
rust

enum TypeExprKind {
    Tensor { shape, dtype, residency: Residency },
    // ...
}

enum Residency {
    Hbm,          // standard; can cross kernel boundaries
    Interior,     // lives only in SMEM/registers within a kernel
}
```

This is an AST-level change. Every pass that touches `TypeExprKind::Tensor` must now handle the new field. In practice this affects a wide swath of passes — type unification, shape inference, effect inference, trait resolution, lowering.

The residency constraint is fundamentally an effect: a kernel that produces an Interior tensor has an Interior effect; a kernel that consumes one must be compatible with that effect. NSL already has a first-class effect system (`EffectExpr` in `decl.rs`) with composition rules.

Expressing residency as an effect:

- Avoids the AST-level field.
- Uses the existing effect-composition machinery.
- Produces better error messages ("effect mismatch: kernel X produces Interior, kernel Y requires Hbm").
- Decouples residency from the tensor's type identity.

§9.2 Proposed revision

Remove the `Residency` enum from `TypeExprKind::Tensor`. Introduce `InteriorEffect` in the effect system. A kernel that produces an interior tensor has the `Interior` effect; a consuming kernel with the `Hbm` effect is incompatible.

The analysis pass that decides CFIP format assignment becomes an effect-inference pass: it reads a kernel's effect signature and decides which format to assign to each interior-effect-tagged tensor.

§9.3 Impact

Code change: ~800 LOC refactor across CFIP, type system, and lowering. No user-visible change; the user still writes `Tensor<[shape], dtype>` without annotating residency. The compiler now infers it.

Paper change: the CFIP paper's §3.4 is replaced with an effect-based formulation. No other papers change.

§9.4 Recommended

Do this before any new paper builds on CFIP's residency. Specifically, the cross-paper connections in this document do not depend on the residency-enum design; they will work with either formulation.

§10 Refinement R-02 — Grad-context bridge behavior for MoE and speculative decoding

§10.1 Issue

The grad-context bridge fix (described in the composition paper §3.14 and referenced in the pending-items list) specifies that `ns1_model_backward` returns input gradients via the grad-context rather than writing them to a caller-owned buffer. The fix works for standard feedforward training.

MoE routing: the forward pass dispatches to a subset of experts per token. Gradients flow back only through the selected experts. The routing gradient (gradient of the routing decision itself) needs special handling. The current grad-context bridge specification does not address this.

Speculative decoding: the forward pass has two paths — a draft model producing candidate tokens and a verify step using the main model. Gradients flow back only through the verified (non-rejected) path. The rejected-path gradients must vanish.

§10.2 Proposed specification

Append a section to the grad-context bridge specification:

MoE routing:

- The grad-context records which experts were dispatched per token.
- During backward, only dispatched-expert gradients are accumulated.
- The routing decision's gradient flows through a straight-through estimator (STE) or a score-function estimator depending on the MoE variant.
- The grad-context exposes a `routing_mask` read API for the optimizer step.

Speculative decoding:

- The grad-context records which verify-step branches were taken (accepted vs rejected).
- During backward, only accepted-branch gradients flow.
- Rejected-branch intermediate tensors are marked no-grad in the grad-context.
- The runtime discards rejected-branch gradient buffers at the verify boundary.

Both cases are compiler-visible — the @moe and @speculative annotations expose the necessary structure.

§10.3 Impact

Code change: ~400 LOC in the grad-context runtime. Integration tests for MoE and speculative decoding backward. The bridge's FFI surface does not change.

Paper change: the grad-context bridge specification (which lives in the project documentation, not a research paper) gains an appendix. If the grad-context bridge ultimately becomes a numbered NSL paper, this material is included from the start.

§10.4 Recommended

Do this before MoE or speculative decoding training is exercised at scale. The current scaffolding likely works for short training runs but is underspecified.

§11 Refinement R-03 — @export and interior quantization formats

§11.1 Issue

The @export specification (A1 + B1) states that exported methods have signatures `(weight_ptrs, num_weights, inputs..., outputs...)` and that boundary dtypes are FP16/FP32 (composition paper invariant I-16). The specification does not address what happens when the exported method internally uses CFIP interior formats, CFTP FP8 weights, or CTQS-dynamic quantization.

For CFIP interior formats: these never cross a kernel boundary, so they never cross the @export boundary. The method's outputs are in FP16/FP32 per invariant I-16. This is fine but should be stated.

For CFTP FP8 weights: the weights are stored in CFTP's FP8+4-bit format. The @export wrapper needs to either (a) serialize the dequant path so the caller sees dequantized weights, or (b) expose the FP8+4-bit format across the @export boundary. Option (a) is simpler but loses the compression benefit; option (b) requires the caller to understand CFTP's format.

For CTQS-dynamic: the quantization schedule changes over training. The exported method's compile target is one specific schedule phase. If the caller uses the exported method in a different phase, results will be wrong.

§11.2 Proposed specification

Append to the @export specification:

CFIP interior formats: Invisible across the @export boundary. The wrapper emits dequant-on-exit as needed.

CFTP FP8 weights: Option (a) — the exported method dequantizes CFTP weights to FP16 before return. For callers who need the FP8 form, a separate `@export(weights_format="cftp")` annotation preserves the native format and the caller is responsible for the dequant.

CTQS schedule: The @export compile target is always the current schedule phase. An @export with CTQS active is pinned to that phase at compile time; callers cannot request a different phase without recompiling. Future work: a CTQS-schedule-aware @export that compiles multiple phases and dispatches at runtime.

§11.3 Impact

Code change: ~300 LOC in the @export wrapper generator. Primarily around the dequant-on-exit logic. Annotation parsing for `weights_format`.

Paper change: the @export design doc (which is not yet a numbered paper) gains a section on quantization interactions. If @export becomes a numbered NSL paper, this is included.

§11.4 Recommended

Do this before CFTP-quantized models are routinely exported. The current under-specification is an accident waiting to happen — the first user who tries to @export a CFTP model will hit undefined behavior.

§12 Refinement R-04 — FASE AdamW bias correction fix elevated to paper appendix

§12.1 Issue

The FASE paper documents the Deferred-mode approach and shows numerical equivalence with standard AdamW. During implementation, an AdamW bias correction bug was caught: the Deferred mode did not produce identical updates to Immediate mode without an explicit bias correction step using `ns1_bias_correction_inv` applied to MHat/VHat via `ScalarMulByBc`. The fix is shipped; the institutional discovery — why is the bias correction non-trivial, and what is the three-part test that proves it works — lives in project notes, not in the FASE paper.

Future implementers modifying the FASE runtime (for example, adding a new optimizer variant) will not see the bias correction invariant unless they read the composition paper's I-03 and the project notes. This is exactly the kind of institutional knowledge that should live in the paper.

§12.2 Proposed addition

Add an appendix to the FASE paper:

Appendix C: AdamW Bias Correction Under FASE Deferred Mode

C.1 The AdamW update rule with bias correction is:

```
m_hat_t = m_t / (1 - beta1^t)
v_hat_t = v_t / (1 - beta2^t)
p_t = p_{t-1} - lr * m_hat_t / (sqrt(v_hat_t) + eps)
```

In FASE Deferred mode, gradients accumulate directly into $\overline{m}_t$ and $\overline{v}_t$ without an intermediate gradient buffer. The bias correction factors $(1 - \beta_1^t)$ and $(1 - \beta_2^t)$ must still be applied.

C.2 Three tests establish equivalence:

- **Test 1 (SGD exact):** FASE Deferred with vanilla SGD must produce bitwise-identical updates to FASE Immediate with vanilla SGD. (No bias correction; pure accumulation.)
- **Test 2 (AdamW FASE-Deferred pipeline):** FASE Deferred AdamW with bias correction must produce numerically equivalent updates to FASE Immediate AdamW across N accumulation steps. Tolerance: 1e-5 relative error per parameter.
- **Test 3 (Jensen fence):** A Jensen's-inequality-based upper bound on the accumulated bias correction error across training; the observed error must never exceed the bound.

C.3 The runtime FFI that implements the bias correction is `ns1_bias_correction_inv` applied to $\overline{m}_{\text{hat}}$ / $\overline{v}_{\text{hat}}$ registers via a `ScalarMulByBc` op. Implementation-level invariants for maintainers:

- The bias correction factor is recomputed each step from the current step count.
- The factor is applied to $\overline{m}_{\text{hat}}$ / $\overline{v}_{\text{hat}}$ after accumulation but before the optimizer step.
- Any optimizer variant (Adam, AdamW, Lion, Shampoo) that uses bias correction must thread through this machinery or explicitly opt out.

§12.3 Impact

Paper change only. No code change (the fix is already shipped). Appendix C occupies ~2 pages.

§12.4 Recommended

Do this when the FASE paper next has an update. This is pure documentation; do not delay other work for it.

§13 Refinement R-05 — WRGA B.3 scaffolding retrospective in the WRGA paper

§13.1 Issue

The WRGA implementation went through a scaffolding phase (B.3) where a PTX scaffolding was built for the fused LoRA kernel. The scaffolding did not include ptxas validation. When the actual kernel emitter was written, several ptxas-level errors appeared that the scaffolding had hidden. The retrospective discovery: PTX emitters must include ptxas validation from the first commit.

This retrospective currently lives in a project file (`project_wrga_ptx_scaffolding_discovered.md`). It is not in the WRGA paper. Future WRGA readers will not see the lesson.

§13.2 Proposed addition

Add a section to the WRGA paper:

Appendix B: Implementation Retrospective — PTX Scaffolding and ptxas Validation

B.1 During implementation, a PTX scaffolding was built without ptxas validation. ptxas-level errors surfaced only when the scaffolding was replaced by the real emitter, causing significant debugging delay.

B.2 The lesson: PTX emitters must include ptxas validation in their first commit. A test suite that validates emitter output against ptxas at build time catches this class of error immediately.

B.3 The 6-commit TDD sequence for the Fused PTX Rewrite (from the design doc) explicitly includes a ptxas validation commit (commit 2: "FA extraction + WRGA LoRA unit ptxas test"). This was instituted in response to the B.3 retrospective.

B.4 Generalized to all future PTX emitters: ptxas validation from commit 1 is a mandatory engineering practice, not an optional test layer.

§13.3 Impact

Paper change only. Appendix B occupies ~1 page.

§13.4 Recommended

Do this when the WRGA paper next has an update. Alongside R-04, this is documentation-only.

§14 Priority matrix and recommended sequencing

The seven compositions and five refinements have different costs and different impacts. The table below summarizes:

ID	Technique	Type	LOC	Sessions	Impact	Priority
C-01	FASE + CFIP memory	Composition	650	1-2	10-20% throughput on threshold models	Tier 1
C-02	CSHA Tier C + CFIP-in-HBM	Composition	1200	3-4	5-10% training throughput	Tier 1
C-03	CPDT + CFIP all-reduce	Composition	1850	6-8	20-40% on multi-node	Tier 2 (multi-node gate)
C-04	WRGA dead-head + CFIP	Composition	900	2-3	1-2%; compounds with pruning	Tier 3
C-05	PCA + CSHA segment-aware	Composition	900	2-3	1-2% on segment-masked layers	Tier 3
C-06	WGGO + CFIP sensitivity	Composition	650	1-2	5-30% accuracy-cost tradeoff	Tier 1
C-07	BYOD + CFIP unification	Composition (refactor)	1650	4-6	Maintainability only	Tier 2
R-01	CFIP effect-based residency	Refinement (code)	800	2-3	Maintainability, better errors	Tier 1
R-02	Grad-context MoE/spec decode	Refinement (code+spec)	400	1-2	Unblocks MoE/spec training	Tier 1
R-03	@export + quantization	Refinement (code+spec)	300	1	Unblocks @export-CFTP	Tier 2
R-04	FASE bias correction appendix	Refinement (doc)	—	—	Doc; institutional memory	Tier 3
R-05	WRGA B.3 retrospective appendix	Refinement (doc)	—	—	Doc; institutional memory	Tier 3

Recommended sequence for Tier 1 work (next 8-12 sessions):

1. R-01 (CFIP effect-based residency) — clean the type-system foundation before building on it.
2. R-02 (grad-context MoE/spec decode) — unblock MoE and speculative decoding.
3. C-06 (WGGO + CFIP sensitivity) — small, immediate accuracy/precision tradeoff win.
4. C-01 (FASE + CFIP memory) — quick throughput win on threshold models.
5. C-02 (CSHA Tier C + CFIP-in-HBM) — larger throughput win on training HBM.

Tier 2 work (sessions 13-24):

6. R-03 (@export + quantization) — unblock @export-CFTP scenarios.
7. C-07 (BYOD unification) — reduce compiler surface area.
8. C-03 (CPDT + CFIP all-reduce) — gated on multi-node test environment.

Tier 3 work (sessions 25+):

9. C-04 (WRGA dead-head) — small gain, only relevant with aggressive pruning.
10. C-05 (PCA + CSHA segment-aware) — small gain on niche workload.
11. R-04, R-05 — documentation-only; do opportunistically.

Total Tier 1+2 scope: approximately 10,400 LOC, 25-35 sessions. Tier 3 is an additional ~1,800 LOC, 4-6 sessions plus documentation.

§15 Testing strategy for cross-technique regressions

Cross-technique compositions introduce a specific class of regression: individual techniques pass their own tests, but a change to one technique breaks the composition. The standard unit-test approach does not catch these; composition-specific tests are required.

Three-tier test strategy:

Tier 1 — Per-composition numerical equivalence tests. For each composition C-XX, a test that verifies: running with the composition enabled produces numerically-equivalent output to running without the composition enabled (within declared tolerance). This is the baseline — the composition must not change results. ~500 LOC per composition, ~3,500 LOC total across seven compositions.

Tier 2 — Composition-matrix tests. A matrix of configurations exercising pairs and triples of compositions simultaneously. For seven compositions there are 21 pairs and 35 triples; we cannot test all. A representative subset of ~10 configurations covers the high-interaction cases (e.g., {C-01, C-02, C-03} for memory + save-activation + all-reduce). ~800 LOC.

Tier 3 — End-to-end training runs. A small model (e.g., 50M parameters) trained for 500 steps with all compositions enabled, convergence metrics compared against a baseline run with no compositions. Run automatically on every composition-related commit. ~400 LOC of test infrastructure plus ~2-4 hours of GPU time per run.

Invariant regression tests. For each of the twenty-three invariants in the composition paper (I-01 through I-23), a test that specifically exercises the invariant. These tests should already exist where the invariants are enforced in code; if any invariant lacks a test, add one. Estimated ~5-10 invariants need a new dedicated test; the remaining ~15-18 are exercised by existing tests.

Total test scope for the compositions: ~5,000 LOC plus automation infrastructure. This is large but not excessive; cross-technique regressions are the hardest class of bug to debug retrospectively, so investment in catching them at commit time pays off.

§16 Conclusion

The cross-paper audit surfaced seven compositional wins and five paper-level refinements. The compositions collectively represent 15-60% training throughput improvement on typical workloads (depending on which compositions apply). The refinements collectively address under-specified behavior that, if left unaddressed, will produce surprising bugs at scale.

None of the compositions requires a new research paper; each is an extension of existing techniques. None of the refinements requires a rewrite; each is a targeted revision. The full program is approximately 12,000 LOC plus tests, implementable in 30-40 sessions.

The three papers in this series (composition, extension directions, connections and refinements) together convert the fourteen individually-complete NSL papers into an integrated system description. The shipped codebase already implements much of this integration; the documentation now catches up.

End of paper.